FunnyFileManager

Architecture Documentation

Terminal-based Dual-Panel File Manager

Inspired by Far Manager

Written in Rust using Ratatui TUI Framework

Version 0.1.0

Table of Contents

1 Introduction 4
1.1 Key Features 4
1.2 Technology Stack 4
2 Project Structure 5
2.1 File Organization 5
2.2 Module Dependency Diagram 5
3 Core Data Structures 6
3.1 Class Diagram 6
3.2 Entity Relationships 7
3.3 Panel State Machine 8
4 User Interface Architecture 9
4.1 Screen Layout Diagram 9
4.2 Layout Constraints 9
4.3 Color Scheme 10
5 Event Loop and Key Handling 11
5.1 Main Event Loop Flowchart 11
5.2 Key Event Priority 12
5.3 Overlay State Machine 12
5.4 Keyboard Shortcuts Reference 13
5.4.1 Navigation Keys 13
5.4.2 Function Keys 13
5.4.3 Quick Search Keys 13
6 File Operation Handlers 14
6.1 Handler Overview 14
6.2 Copy Operation Sequence Diagram 15
6.3 Move Operation Sequence Diagram 16
6.4 Command Line Commands 17
7 Utility Functions 18
7.1 utils.rs Overview 18
7.2 format_size() Logic 18
7.3 unique_destination() Algorithm 18
7.4 Recursive copy_dir_all() Algorithm 19
8 Editor Integration 20
8.1 Editor Selection Strategy 20
8.2 handle_edit() Sequence 21
9 Data Flow Patterns 22
9.1 Panel Navigation Flow 22
9.2 Quick Search Flow 23
10 Error Handling 24
10.1 Error Handling Strategy 24
10.2 Error Flow Diagram 24
11 Testing Architecture 25
11.1 Test Pyramid 25
11.2 Test Infrastructure 25
11.3 Mutation Testing Points 25
12 Architectural Decisions 26

12.1 Key Design Choices 26
12.1.1 1. Dual-Panel Architecture 26
12.1.2 2. Overlay Stack Pattern 26
12.1.3 3. Selection Preservation 26
12.1.4 4. Rename-with-Fallback for Move 26
12.1.5 5. Unique Destination Naming 26
13 Extension Guide 27
13.1 Adding New Features 27
13.1.1 New F-Key Command 27
13.1.2 New Overlay 27
13.1.3 New Sort Mode 27
13.2 Configuration Points 27
14 Conclusion 28
14.1 Architectural Highlights 28

1 Introduction

FunnyFileManager (FFM) is a terminal-based dual-panel file manager written in Rust. It provides a Far Manager-style user interface with keyboard-driven navigation and file operations.

1.1 Key Features

- **Dual-panel layout** for efficient file operations between directories
- **Far Manager color scheme** (dark blue background, cyan text)
- **F-key commands** for all major operations
- **Quick search** with Alt+letter combinations
- **Bookmarks** for frequently visited directories
- **Built-in file viewer** for text files
- **External editor integration** with VS Code priority

1.2 Technology Stack

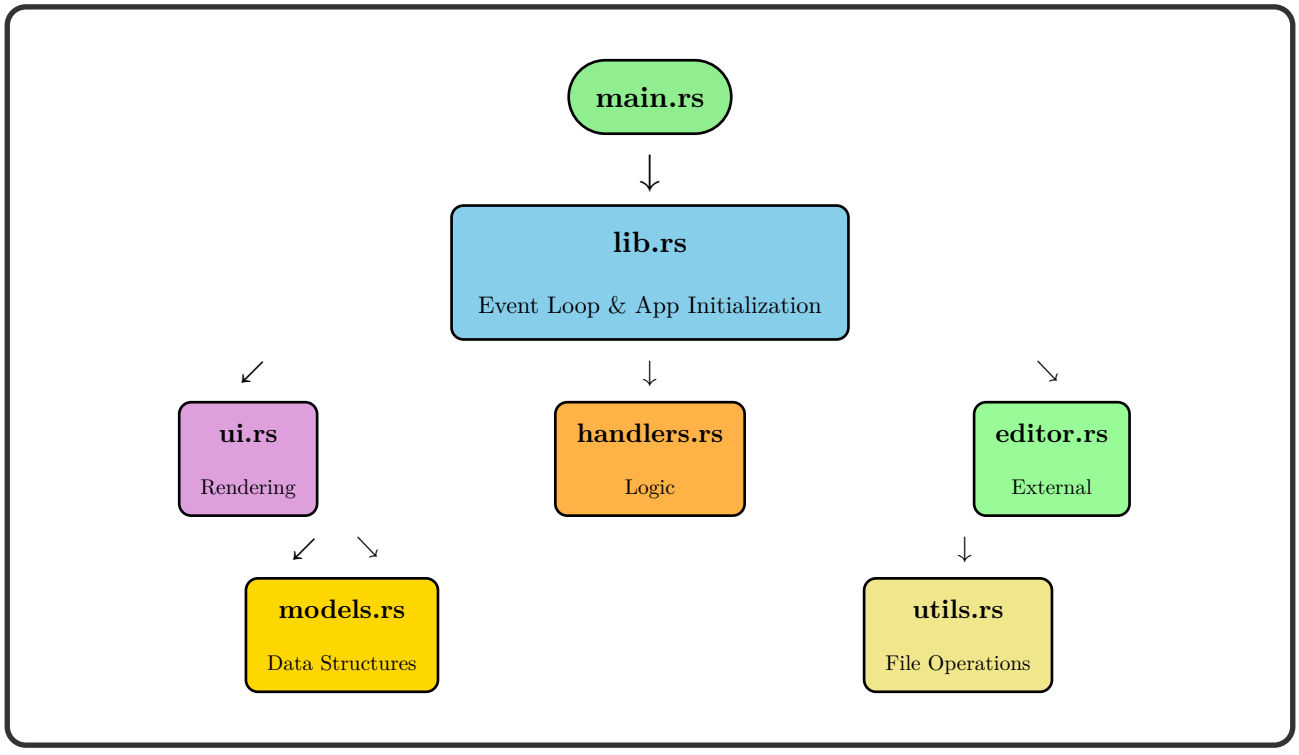
Component	Technology
Language	Rust 2021 Edition
TUI Framework	Ratatui 0.26
Terminal Backend	Crossterm 0.27
Testing	Built-in + Tarpaulin for coverage

2 Project Structure

2.1 File Organization

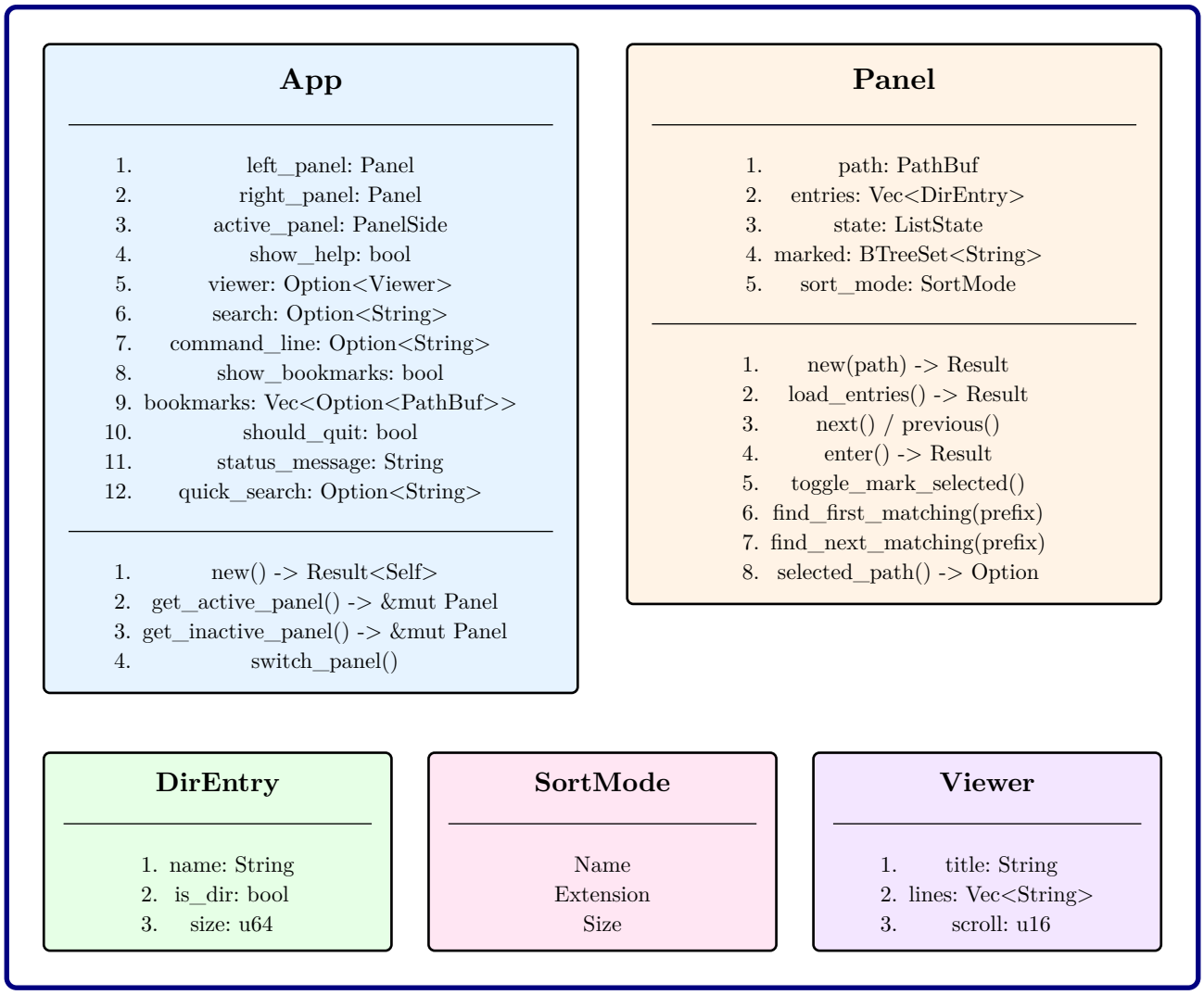
```
FunnyFileManager/
├── src/
│   ├── main.rs          # Entry point (thin wrapper)
│   ├── lib.rs           # Public API, event loop
│   ├── models.rs        # Core data structures
│   ├── ui.rs            # TUI rendering
│   ├── handlers.rs      # Event handlers
│   ├── utils.rs         # Helper functions
│   └── editor.rs        # Editor integration
├── tests/
│   ├── editor_test.rs
│   ├── handlers_test.rs
│   └── navigation_features_test.rs
├── Cargo.toml
└── CLAUDE.md
```

2.2 Module Dependency Diagram

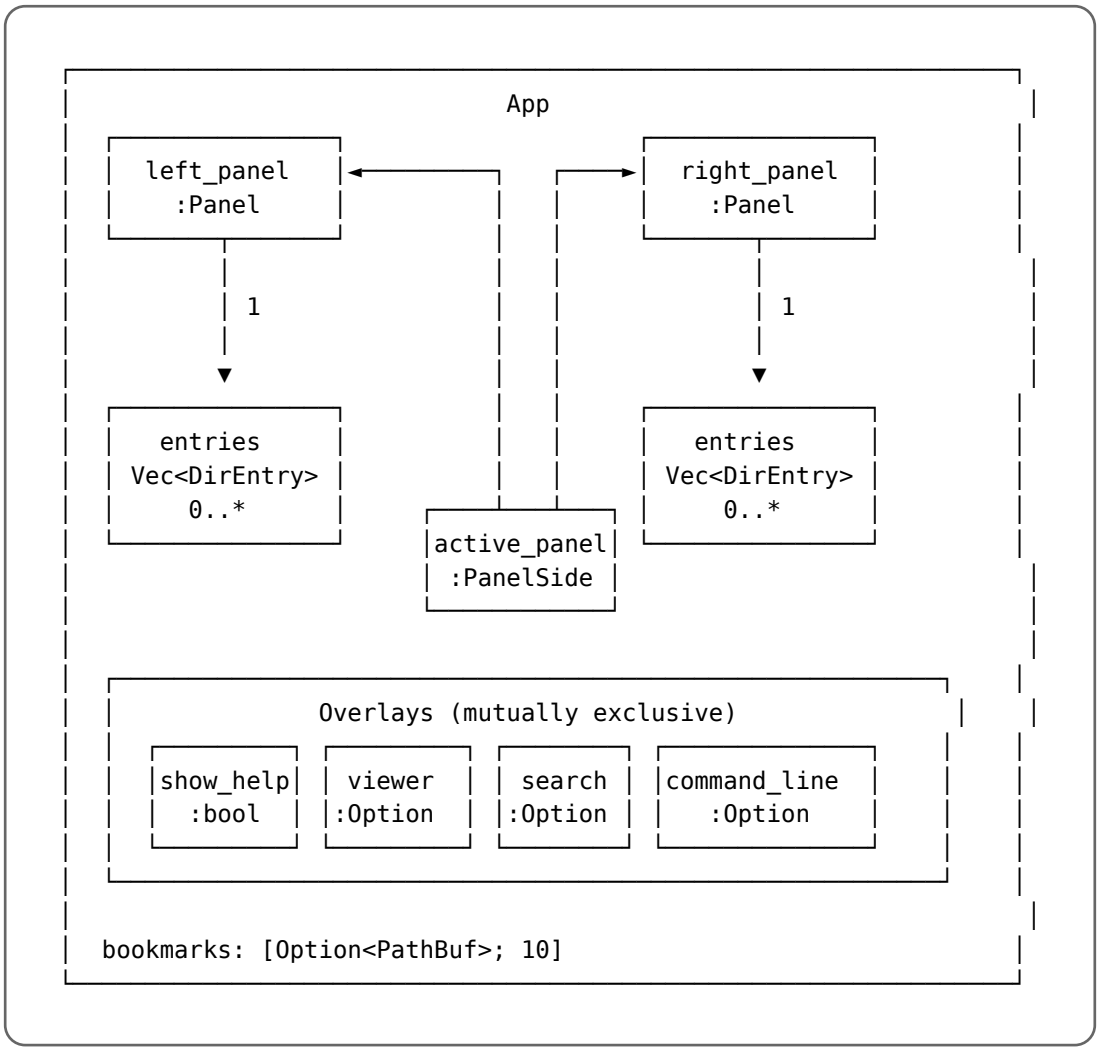


3 Core Data Structures

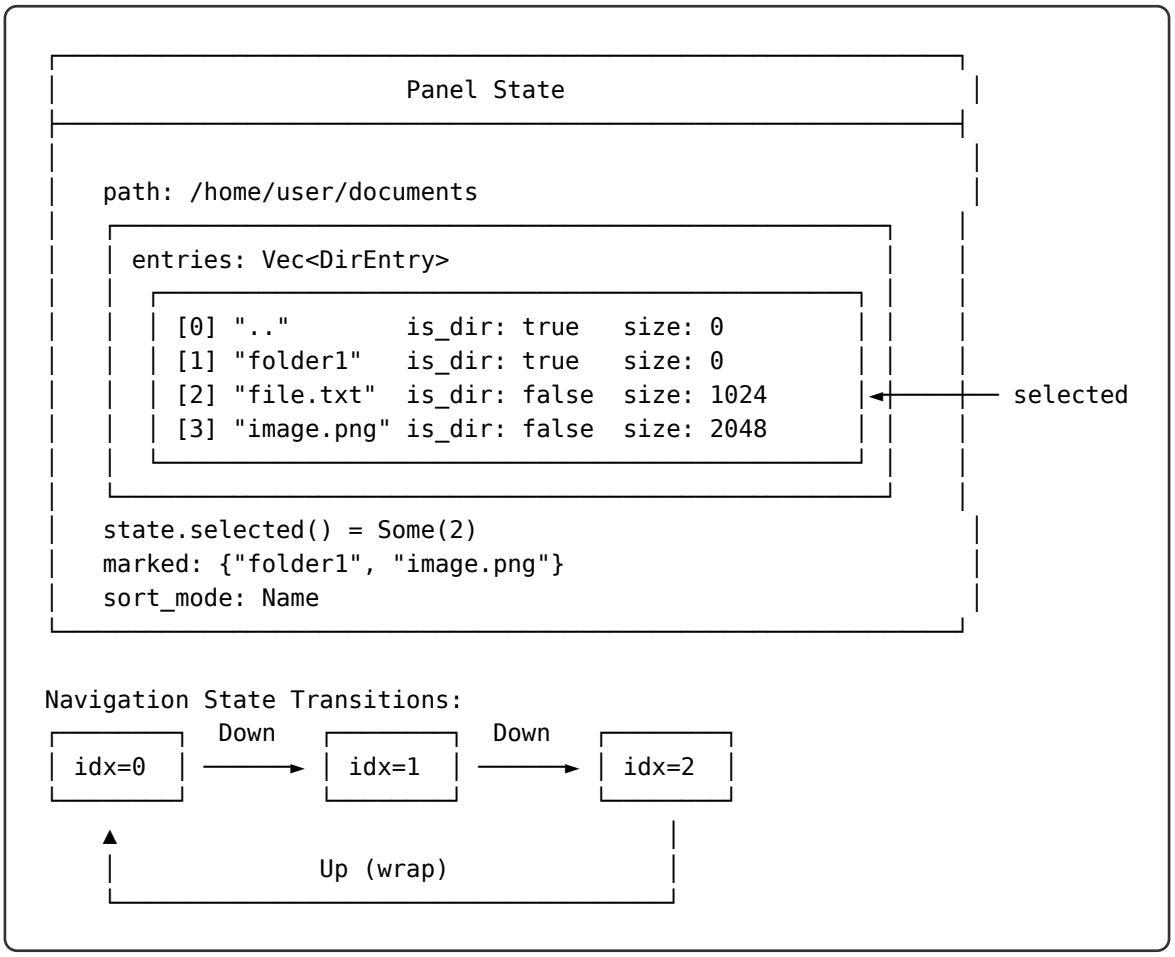
3.1 Class Diagram



3.2 Entity Relationships

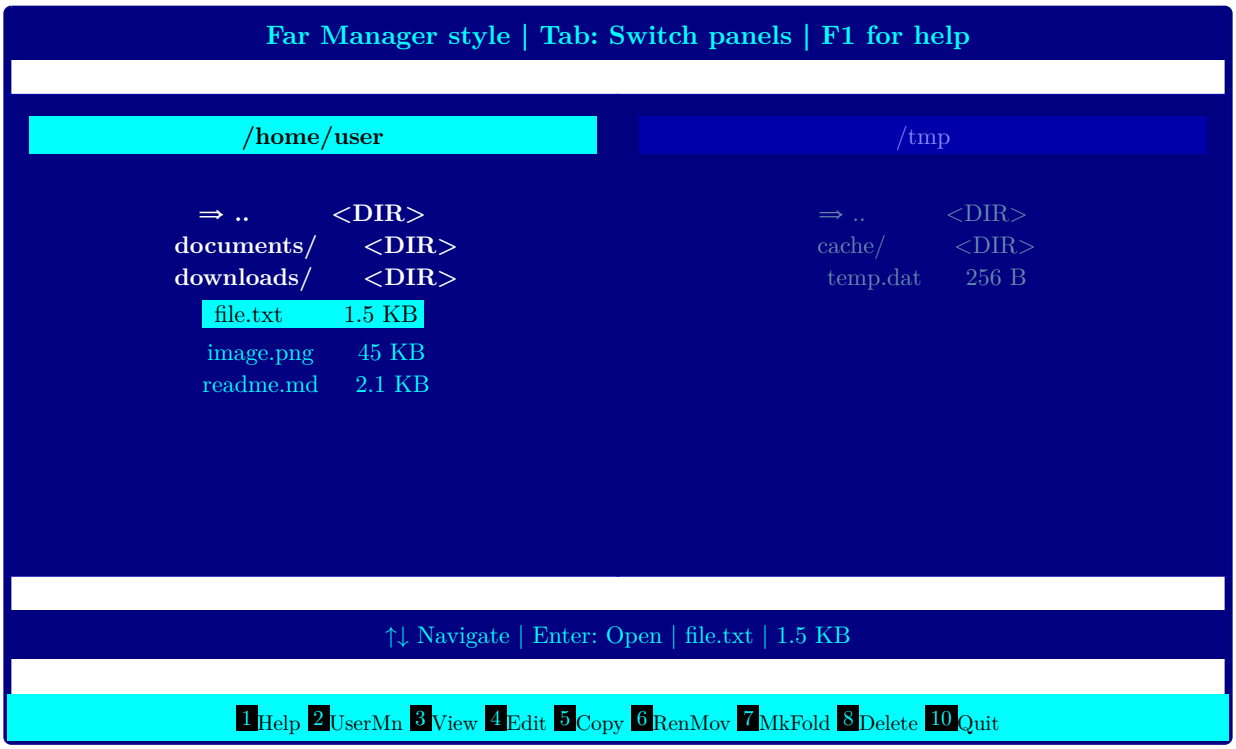


3.3 Panel State Machine



4 User Interface Architecture

4.1 Screen Layout Diagram



4.2 Layout Constraints

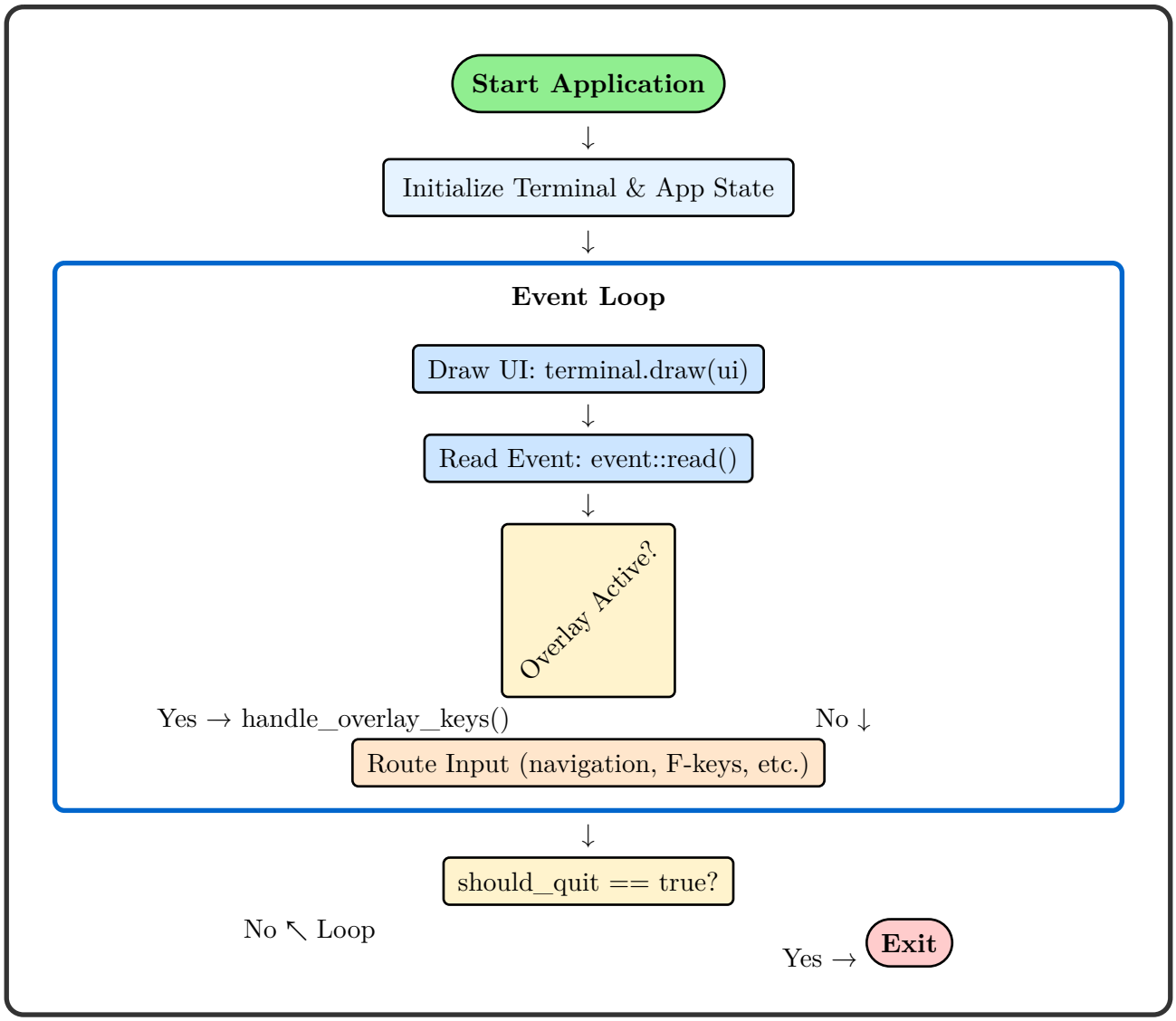
Component	Height	Description
Header	1 line	Application title and keyboard hints
Panels	Remaining	50/50 horizontal split
Status Bar	2 lines	File info and quick search display
Command Bar	2 lines	F-key function labels

4.3 Color Scheme

Element	Color	RGB Value
Background	Far Blue	RGB(0, 0, 128)
Active Border	Cyan Bold	RGB(0, 255, 255)
Inactive Border	Dark Blue	RGB(0, 0, 139)
Directories	White Bold	RGB(255, 255, 255)
Files	Cyan	RGB(0, 255, 255)
File Sizes	Light Cyan	RGB(224, 255, 255)
Selection	Black on Cyan	BG: Cyan, FG: Black
Marked Files	Yellow	RGB(255, 255, 0)

5 Event Loop and Key Handling

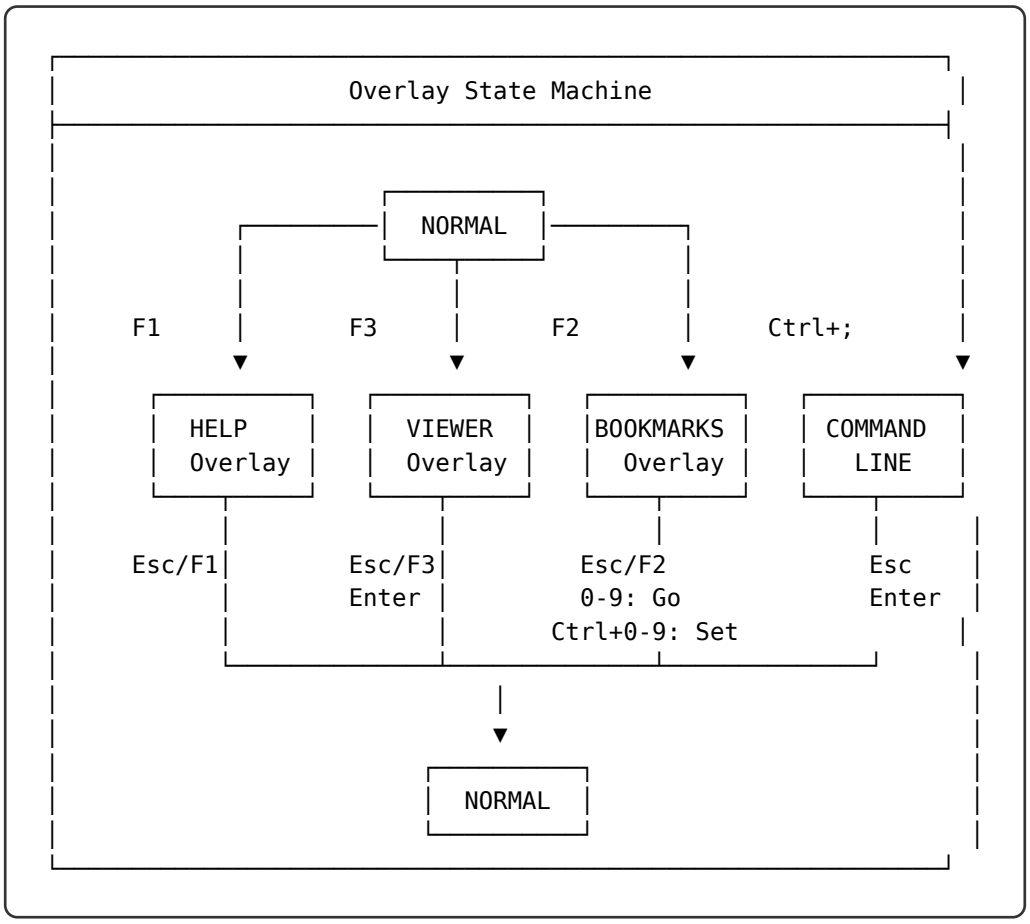
5.1 Main Event Loop Flowchart



5.2 Key Event Priority

Priority	Handler	Keys Handled
1	Overlay Keys	Viewer, Help, Search, Bookmarks, Command Line
2	Quit Keys	Q, Ctrl+C, F10
3	Navigation	Up, Down, Enter, Tab, Left, Right
4	Function Keys	F1-F8 for file operations
5	Marking	Insert, Space for file selection
6	Quick Search	Alt+letter for fast navigation
7	Other Keys	Clears quick search state

5.3 Overlay State Machine



5.4 Keyboard Shortcuts Reference

5.4.1 Navigation Keys

Key	Action
↑ / ↓	Move selection up/down (with wraparound)
← / →	Switch between panels
Tab	Toggle between panels
Enter	Enter directory or go back (..)
Home	Jump to first item
End	Jump to last item
PgUp / PgDn	Page navigation

5.4.2 Function Keys

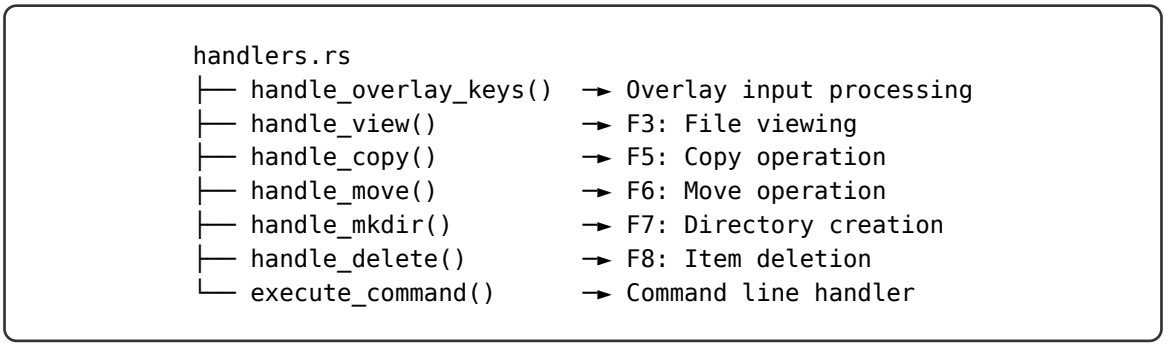
Key	Command	Description
F1	Help	Toggle help overlay
F2	Bookmarks	Show bookmark manager
F3	View	View file in built-in viewer
F4	Edit	Open file in external editor
F5	Copy	Copy to other panel
F6	Move	Move to other panel
F7	Mkdir	Create new directory
F8	Delete	Delete selected item
F10	Quit	Exit application

5.4.3 Quick Search Keys

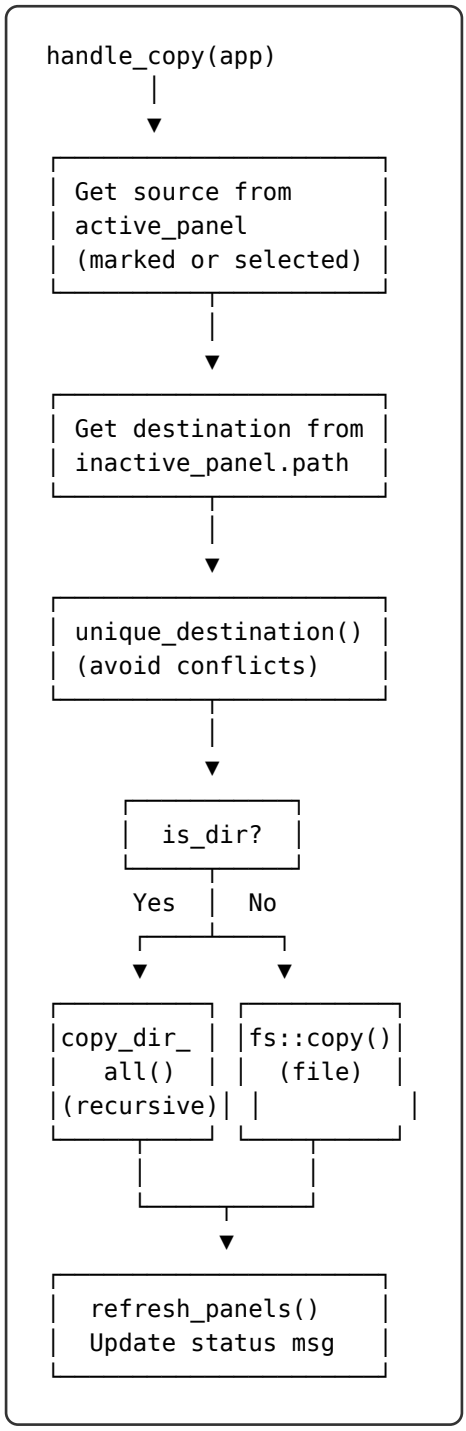
Key	Action
Alt+letter	Add letter to search query, jump to first match
Backspace	Remove last character or clear search
Esc	Clear quick search

6 File Operation Handlers

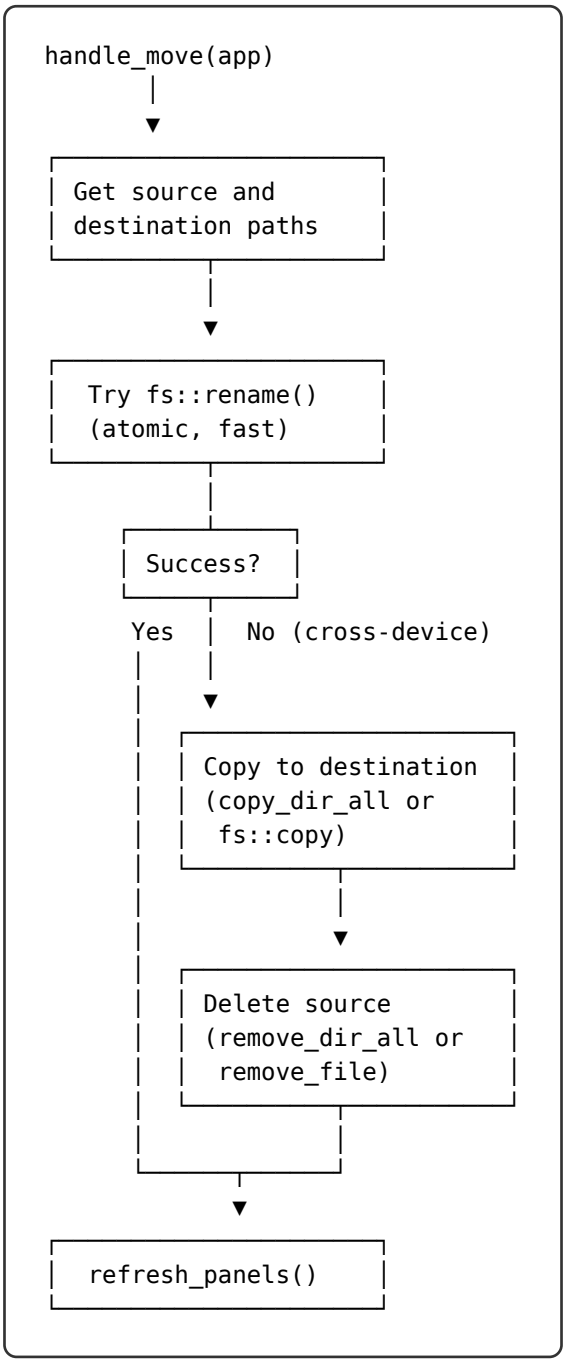
6.1 Handler Overview



6.2 Copy Operation Sequence Diagram



6.3 Move Operation Sequence Diagram



6.4 Command Line Commands

Command	Arguments	Action
`cd`	`<path>`	Change directory (active panel)
`pwd`		Show current path in status
`sort`	`name|ext|size`	Change sort mode
`bookmark`	`set|go <0-9>`	Manage bookmarks
`mark`	`all`	Mark all files
`unmark`		Clear all marks
`help`		Show help overlay
`quit`		Exit application

7 Utility Functions

7.1 utils.rs Overview

Function	Purpose
`format_size(bytes)`	Convert bytes to human-readable format
`unique_destination(base, name)`	Generate unique filename if exists
`copy_dir_all(src, dst)`	Recursive directory copy
`refresh_panels(app)`	Reload both panels

7.2 format_size() Logic

```
format_size(bytes: u64) → String

bytes < 1024           →  "{bytes} B"
bytes < 1,048,576      →  "{bytes/1024:.2} KB"
bytes < 1,073,741,824  →  "{bytes/1048576:.2} MB"
bytes >= 1,073,741,824 →  "{bytes/1073741824:.2} GB"

Examples:
  512        →  "512 B"
  2048       →  "2.00 KB"
  5242880    →  "5.00 MB"
  2147483648 →  "2.00 GB"
```

7.3 unique_destination() Algorithm

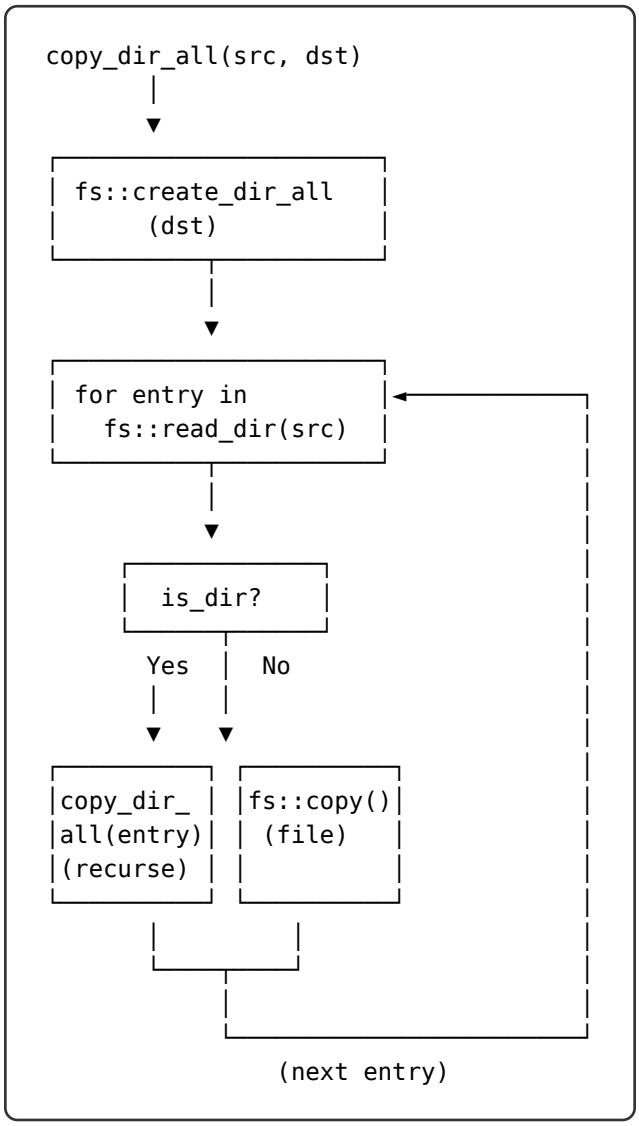
```
unique_destination(base: &Path, file_name: &OsStr) → PathBuf

Input: base=/target, file_name="test.txt"

Check: /target/test.txt exists?
  No  → return /target/test.txt
  Yes → try /target/test_1.txt
          → try /target/test_2.txt
          → try /target/test_3.txt
          → ... until unique found

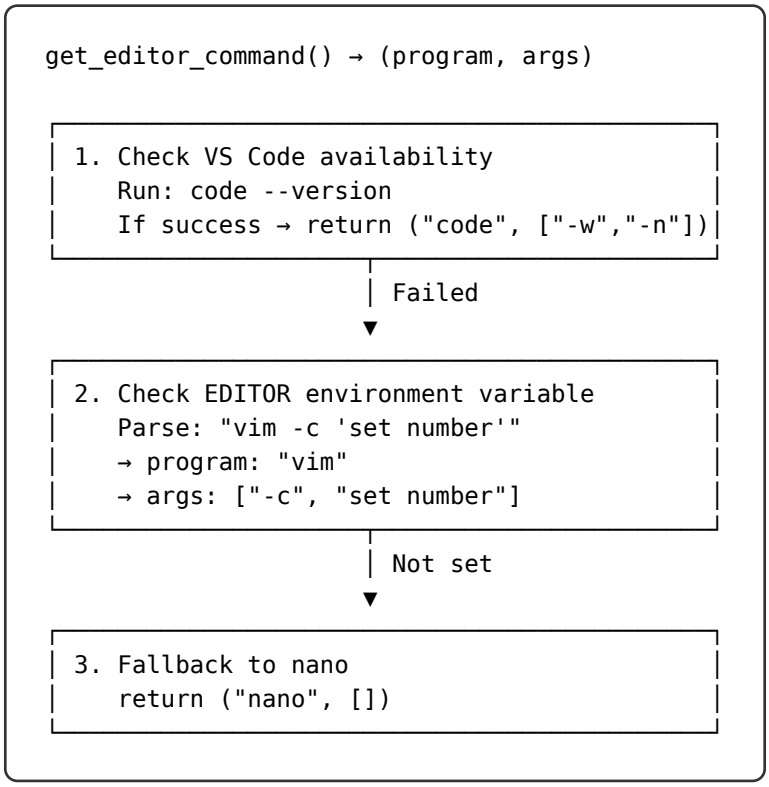
Edge cases handled:
  • Dotfiles: ".env" → ".env_1"
  • No extension: "Makefile" → "Makefile_1"
  • Multiple dots: "file.tar.gz" → "file.tar_1.gz"
```

7.4 Recursive copy_dir_all() Algorithm

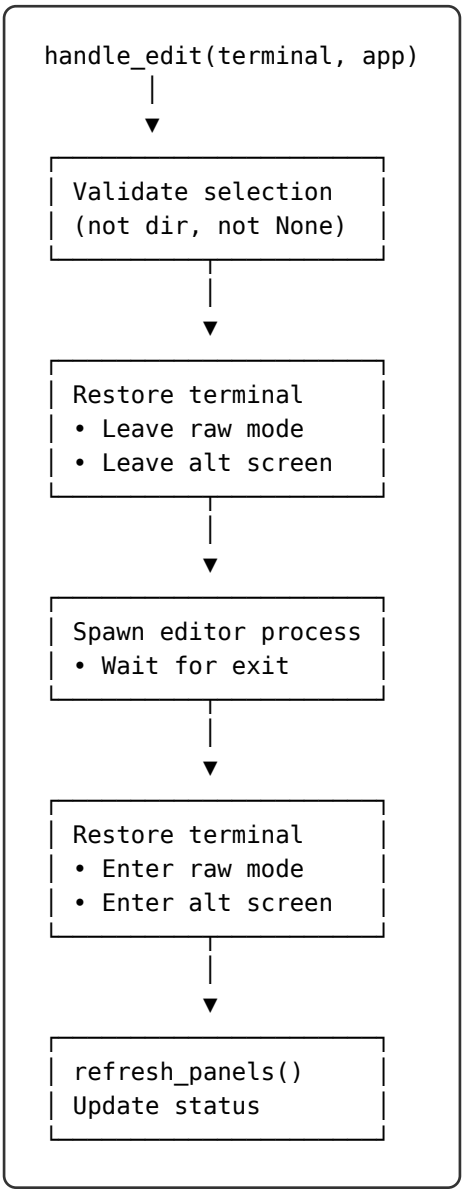


8 Editor Integration

8.1 Editor Selection Strategy

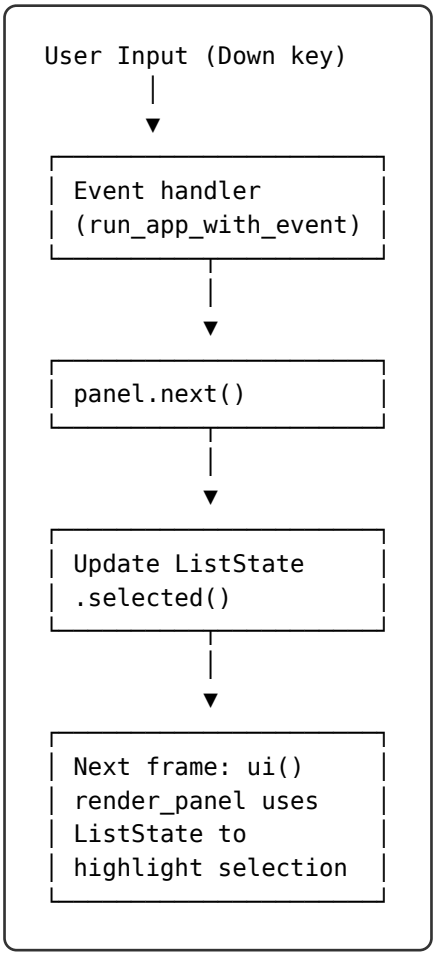


8.2 handle_edit() Sequence

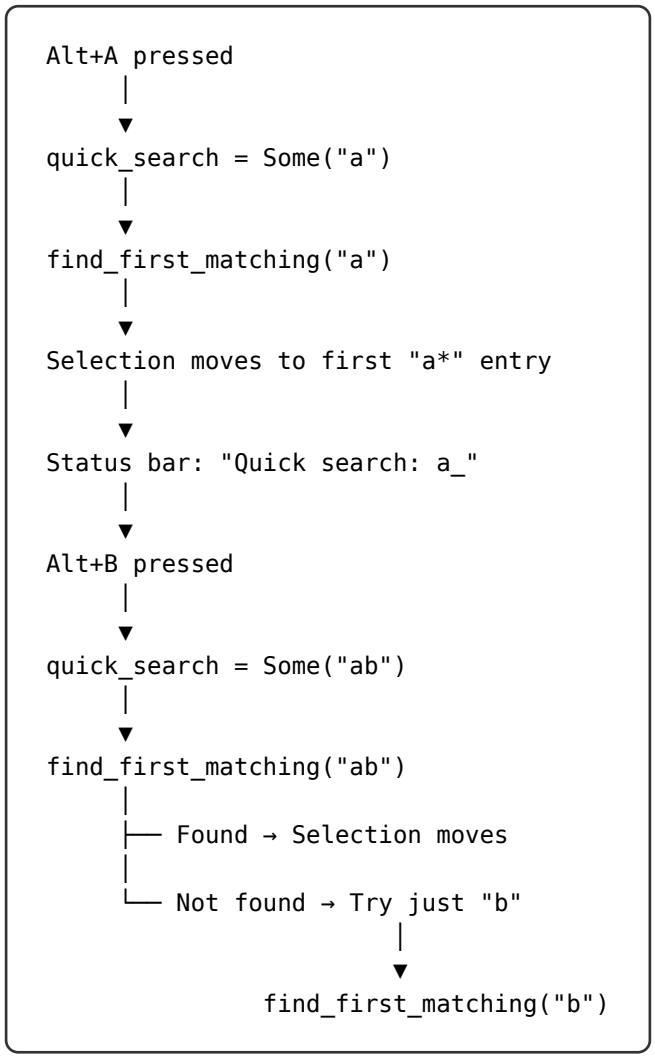


9 Data Flow Patterns

9.1 Panel Navigation Flow



9.2 Quick Search Flow

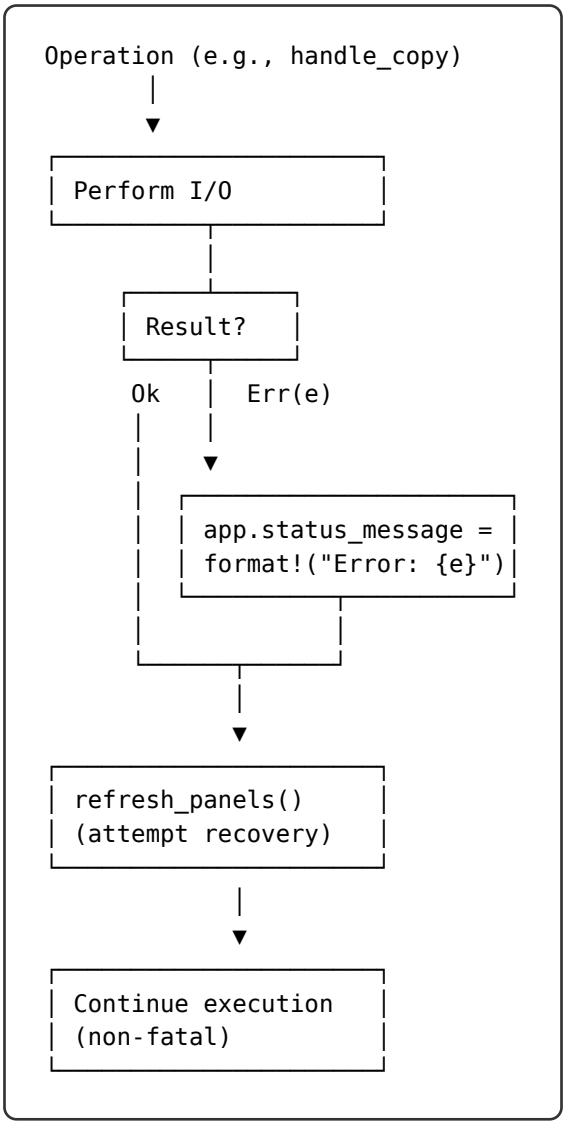


10 Error Handling

10.1 Error Handling Strategy

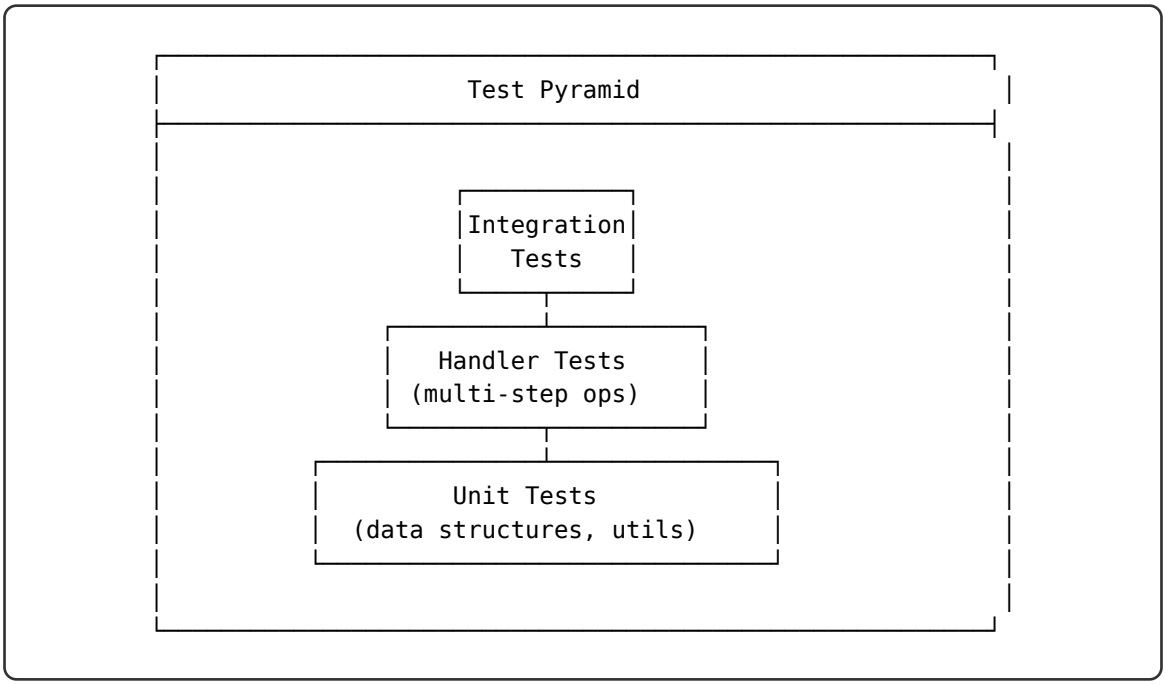
Operation	Error Type	Handling
Directory access	`io::Error`	Show in status, continue
File view (non-UTF8)	`io::Error`	Display error message
Copy/Move/Delete	`io::Error`	Status message, refresh panels
Editor launch	Process error	Status: “Failed to launch”
Panel reload	`io::Error`	Attempt refresh anyway

10.2 Error Flow Diagram



11 Testing Architecture

11.1 Test Pyramid



11.2 Test Infrastructure

Component	Purpose
`WritableTestBackend`	Custom ratatui backend for testing
`tempfile::TempDir`	Temporary directories for FS tests
`FORCE_RENAME_FAILURE`	Simulate cross-device moves
`FORCE_COPY_DIR_ALL_STEP`	Inject errors at specific steps
`FORCE_REFRESH_ERROR`	Test panel refresh failures

11.3 Mutation Testing Points

The codebase includes specific injection points for mutation testing:

```
// In copy_dir_all()
#[cfg(any(test, tarpaulin))]
if FORCE_COPY_DIR_ALL_STEP.load(Ordering::SeqCst) == STEP {
    return Err(io::Error::new(...));
}

// In handle_move()
#[cfg(any(test, tarpaulin))]
static FORCE_RENAME_FAILURE: AtomicBool = AtomicBool::new(false);
```

12 Architectural Decisions

12.1 Key Design Choices

12.1.1 1. Dual-Panel Architecture

Rationale: Enables intuitive source-target operations without dialogs.

Benefits:
- Visual directory comparison
- One-key file operations (F5, F6)
- Independent navigation state per panel

12.1.2 2. Overlay Stack Pattern

Rationale: Modal interactions without complex window management.

Benefits:
- Simple state machine (one overlay at a time)
- Consistent Escape behavior
- Easy to add new overlays

12.1.3 3. Selection Preservation

Rationale: Reduces user disorientation after operations.

Implementation:
- Save selected filename before reload
- Find filename after reload
- Fall back to first item if not found

12.1.4 4. Rename-with-Fallback for Move

Rationale: Handle cross-filesystem moves correctly.

Strategy:
1. Try atomic `fs::rename()` first (fast path)
2. On failure, copy then delete (slow path)
3. User sees same result regardless of filesystems

12.1.5 5. Unique Destination Naming

Rationale: Never silently overwrite existing files.

Strategy:
- Check if destination exists
- Append `_1`, `_2`, etc. to base name
- Preserve original extension

13 Extension Guide

13.1 Adding New Features

13.1.1 New F-Key Command

```
// In lib.rs, match key.code block:
KeyCode::F(9) => {
    handle_new_feature(&mut app)?;
}
```

13.1.2 New Overlay

```
// In models.rs, add to App:
pub new_overlay: Option<NewOverlayData>,

// In handlers.rs, handle_overlay_keys():
if let Some(ref mut overlay) = app.new_overlay {
    // Handle overlay-specific keys
    return true;
}

// In ui.rs, render_overlays():
if app.new_overlay.is_some() {
    render_new_overlay(frame, app);
}
```

13.1.3 New Sort Mode

```
// In models.rs, SortMode enum:
pub enum SortMode {
    Name,
    Extension,
    Size,
    Date,  // New mode
}

// Add comparison logic in Panel::load_entries()
```

13.2 Configuration Points

Setting	Location	Current Value
Initial directory	`lib.rs`	`env::current_dir()`
Color scheme	`ui.rs`	Far Manager blue
Editor preference	`editor.rs`	VS Code > EDITOR > nano
Bookmark slots	`models.rs`	10 slots (0-9)
Debounce time	`lib.rs`	250ms (coverage mode)

14 Conclusion

FunnyFileManager demonstrates a well-architected terminal application with:

14.1 Architectural Highlights

Separation of Concerns	Distinct modules for models, UI, handlers, utilities
Immutable Rendering	UI functions don't mutate state
Comprehensive Error Handling	Non-fatal errors, graceful recovery
Far Manager UX	Familiar hotkeys and color scheme
Testable Design	Injection points for mutation testing
Terminal Safety	Proper mode restoration for external programs

FunnyFileManager

Modern Terminal File Manager in Rust

Built with Ratatui + Crossterm